

Weather-Adaptive Task Allocation for Vehicle Fleets in Disaster Relief Operations: A Multi-Scenario Optimization Framework

Omar ElHarridy*, Mohamed Elmenshawhy*, Ahmed Gasser*, Mahmoud Hebishy*, Abdelrahman Sayed†, Aly Ibrahim†,

Jessica Magdy†, Omar Shehata†

**Computer Science and Engineering Department, Faculty of Media Engineering and Technology, German University in Cairo, Egypt*

†Mechatronics Department, Faculty of Engineering and Materials Science, German University in Cairo, Egypt
{omar.elharridy, mohamed.elmenshawhy, ahmed.gasser, mahmoud.hebishy, abdelrahman.sayed, aly.ibrahim}
}@student.guc.edu.eg, {jessica.gergis, omar.mohamad}@guc.edu.eg

Abstract—Multi-cooperative Vehicle Fleets have been the subject of many research papers and rigorous tests over the years. In this paper we aim to implement a novel multi-level optimization architecture, that optimizes both path and task allocation through the use of state-of-the-art optimization techniques and algorithms on Disaster relief operations. This architecture takes into account Weather Conditions, like Wildfires, Blizzards, Floods, and many more. Due to the severity and sensitivity of the context, it is very crucial that the applied Optimization techniques take into account these weather conditions and their effects on a vehicle fleets' Speed, fuel consumption, distance traveled, task allocation, chosen path and the overall success of the operation. The main algorithms covered in this paper are Simulated Annealing (SA), Genetic Algorithm (GA), Discrete Grey Wolf and Artificial Bee Colony (ABC).

Index Terms—SA, GA, Discrete Grey Wolf, ABC

I. INTRODUCTION

The importance of the multi-cooperative vehicle fleet problem has been extensively addressed in terms of routing, task allocation, and operational efficiency, among others. The relevance of such a problem is even more emphasized in disaster relief scenarios, considering that such engagements often rely on timely, efficient, and effective coverage of disaster relief intervals. Unlike the traditional vehicle fleet problem, disaster-relief scenarios often experience unexpected and dynamic environmental factors, such as weather conditions. In this paper, we address a disaster relief scenario that includes the deployment of multiple vehicles from different stations to allocate tasks, all relating to specific assignments or areas of interest. The objectives of this model are two-fold: minimizing distance traveled by these vehicles through optimized route planning and maximizing the number of tasks accomplished by these vehicles through task allocation.

There are several factors to be considered in this optimization problem. Firstly, the weather conditions on the map depend on different points and directly affect the speed of the vehicles, and thus the efficiency of the mission. Secondly, we have different vehicles at different speeds.

The model must work under various operational limitations. For example, each vehicle has a limited distance to travel due to fuel capacity. The tasks executed by each vehicle must be exclusive, implying each task can be assigned to no more than

one vehicle. Finally, each vehicle must avoid spatial-temporal collisions, which implies each vehicle can only be at one location at any time. The tasks executed by each vehicle follow a predetermined order of priority.

To handle this challenging problem of constrained optimization, we examine different advanced techniques of meta-heuristic optimization and their effectiveness in improving operational efficiency with different weather.

This paper is organized as follows: Section II reviews related work. Section III presents the problem formulation. Section IV describes the implementation of the following applied meta-heuristic optimization techniques: Simulated Annealing [1], chosen as a trajectory-based method with a well-understood exploration-exploitation trade-off; Genetic Algorithms [2], selected for their population-based global search via evolutionary operators; Discrete Grey Wolf Optimizer [3], included as a swarm-intelligence approach with competitive performance on discrete scheduling problems; and Artificial Bee Colony [4], providing a distinct foraging-inspired search mechanism within a multi-layer framework for joint task allocation and route optimization. These four algorithms were selected to enable a cross-paradigm comparison across complementary search strategies. Section V reports the experimental results. Finally, Section VI concludes the paper.

II. LITERATURE REVIEW

To solve complex routing problems efficiently, hierarchical architectures have been widely explored. For instance, a hierarchical framework decoupling strategic fleet sizing from tactical task allocation was developed in [5]. Using a Grey Wolf Optimizer (GWO) nested with a Consensus-Based Bundle Algorithm (CBBA), this approach minimizes the global scheduling cost. This hierarchical structure mirrors our architecture but uses consensus for conflict resolution, whereas our model enforces strict physical collision avoidance in a shared grid. Extending robust planning to adaptive execution, recent models have combined Distributionally Robust Optimization (DRO) for offline planning with Contextual Bandits (CB) for online adaptive dispatch in disaster medical logistics [6]. This coordinates heterogeneous fleets of ambulances, drones, and robots to navigate uncertain demand,

infrastructure degradation, and time-sensitive deliveries. To address the suboptimality of classical decoupled approaches, Dubins path constraints have been directly integrated into task allocation, replacing Euclidean distance with actual path length in the cost metric [7]. Furthermore, electric vehicle routing under uncertain demand and dynamic traffic conditions has been addressed through robust optimization [8], demonstrating that while robust solutions may have higher objective values than deterministic models, they ensure greater reliability.

Within these hierarchical structures, coordinating multi-agent and heterogeneous fleets introduces additional complexity. A high-efficiency task allocation algorithm for offshore inspections was proposed in [9], balancing energy consumption and total mission time using an Improved Ant Colony Optimization (IACO) algorithm. While addressing cooperative allocation, the model relies on open-space assumptions. Our work argues that in disaster relief, “friction” is not just energy loss but physical collision risk, requiring stricter grid-based occupancy constraints. To handle heterogeneous fleets in post-disaster rescue, a cooperative framework combining aerial agility with ground payload capacity has demonstrated significant efficiency improvements in debris fields [10]. This directly supports our hypothesis that heterogeneous capabilities must be modeled explicitly to optimize real-world relief operations. Managing task transitions and modal shifts is equally critical to fleet efficiency, as addressed by the “Rich Vehicle Routing Problem” (RVRP) [11]. By explicitly quantifying the “Context Switching Tax” and assigning penalties to transfers between vehicle modes, this work provides the theoretical basis for our focus on “switching costs,” validating that minimizing transition friction is as critical as minimizing distance in disaster logistics.

Similarly, multi-agent coordination in structured grid environments forms the baseline for complex routing operations. Before tackling unpredictable post-disaster environments, foundational UGV route optimization has been heavily explored in deterministic urban settings, such as multi-UGV trajectories for municipal waste collection, establishing baseline efficiencies for grid-based fleet operations [12]. Building on this structured coordination, metaheuristic optimization has been adapted for dynamic search and rescue missions [13]. A multi-AGV path planning framework was developed to maximize exploration coverage while minimizing overall mission completion time under battery, spacing, and collision avoidance constraints. AGV path planning on a discrete grid was optimized using SA, GA, PSO, and APO, with APO minimizing time and GA balancing performance and cost, but without weather-dependent dynamics.

Beyond fleet coordination, executing these allocations in dynamic environments requires incorporating weather constraints into optimization models. The critical failure point of ignoring dynamic wind conditions in standard logistics models was addressed through a multi-objective optimization framework that minimizes energy consumption and delivery delays [14]. Furthermore, it has been demonstrated that wind-sensitive stochastic optimization outperforms deterministic methods in volatile conditions [15]. Similarly, an energy-efficiency model for Urban Air Mobility (UAM) has been

developed to account for spatial wind gradients by dynamically optimizing velocity profiles [16]. This highlights the importance of variable velocity in optimization, a concept we mirror by using weather-dependent velocity multipliers for our ground vehicles. Addressing environmental resilience directly, Artificial Potential Fields (APF) have been integrated with evolutionary optimization to create wind-resilient trajectories [17]. While effective, this approach is limited to single-agent scenarios, failing to address the “Context Switching Tax” of multi-task allocation. The limitation of weather blindness in standard A* planners has also been addressed by constructing heuristic functions that integrate real-time wind field data [18]. This confirms that static heuristics are insufficient for dynamic environments, supporting our use of dynamic velocity multipliers for accurate path cost estimation.

To operationalize these sophisticated models, the underlying optimization techniques must draw upon heavily validated metaheuristics. Our comparative framework benchmarks these complex, weather-adaptive formulations against standard Simulated Annealing [1], Genetic Algorithms [2], the Discrete Grey Wolf Optimizer [3], and Artificial Bee Colony algorithms [4] to establish baseline efficiency and validate our performance gains.

III. PROBLEM FORMULATION

A. Decision Variables and Representation

Let c be the number of cars and t the number of tasks. The assignment variable $x_{ij} \in \{0, 1\}$ indicates whether car i is assigned to task j , $\forall i \in \{1, \dots, c\}$, $j \in \{1, \dots, t\}$. For each active assignment, the path $\mathbf{p}_{ij} = \{l_{ij}^1, l_{ij}^2, \dots, l_{ij}^{n_{ij}}\}$ is the ordered sequence of traversed cells. A solution is encoded as a $c \times t$ matrix $\mathbf{S}$ with $S_{ij} = \mathbf{p}_{ij}$ if $x_{ij} = 1$ and $S_{ij} = \emptyset$ otherwise.

B. Objectives

We maximize completed tasks and minimize total travel distance:

$$\begin{aligned} \max f_1(\mathbf{S}) &= \sum_{i=1}^c \sum_{j=1}^t x_{ij}, \\ \min f_2(\mathbf{S}) &= \sum_{i=1}^c \sum_{j=1}^t \sum_{k=1}^{n_{ij}-1} \sqrt{(x_{k+1} - x_k)^2 + (y_{k+1} - y_k)^2}. \end{aligned} \quad (1)$$

We use weighted scalarization:

$$\min F(\mathbf{S}) = w_1(t - f_1(\mathbf{S})) + w_2 f_2(\mathbf{S}), \quad (2)$$

Maximizing f_1 is recast as minimizing uncompleted tasks $(t - f_1(\mathbf{S}))$, unifying both objectives under a single minimization.

with $w_1 = 100.0$ and $w_2 = 1.0$, chosen to prioritize task completion over travel distance, as validated by sensitivity analysis over $w_1 \in [50, 200]$.

C. Weather Conditions

Grid cells may carry weather effects that reduce car speed. Three scenario complexities are considered: **Low** (Storm 60%, Wind 85%), **Medium** (Fog 50%, Snow 70%), and **High** (Sand 40%, Wind 65%, Fog 55%).

D. Constraints

- 1) **Unique Task Assignment:** $\sum_{i=1}^c \mathbb{1}(\|S_{ij}\| > 0) \leq 1, \forall j.$
- 2) **Maximum Distance:** $\sum_{j=1}^t \text{Dist}(S_{ij}) \leq D_i^{\max}, \forall i.$
- 3) **Collision Avoidance:** $l_k^i = l_m^j \Rightarrow t_k^i \neq t_m^j, \forall i \neq j,$ where travel time accounts for the weather-dependent velocity multiplier $M(c_{k+1})$.

E. Assumptions

- Each cell carries at most one weather condition.
- Cars are not assigned tasks beyond their maximum reachable distance.
- Starting positions of cars and task locations are spatially distinct.
- Each task has a unique priority.

IV. METHODOLOGY

This Section will cover all the various Optimization Algorithms and Techniques employed on our problem.

A. Simulated Annealing

1) Algorithm Components: Temperature Schedules:

$$T_i = T_0 \cdot \alpha^i \quad (\text{Geometric cooling}) \quad (3)$$

$$T_i = T_0 + \beta \cdot i \quad (\text{Linear cooling}) \quad (4)$$

where $\alpha = 0.95$ is the cooling rate and $\beta = \frac{T_f - T_0}{i_{max}}$.

Metropolis Acceptance Criterion:

$$P(\text{accept}) = \begin{cases} 1, & \text{if } \Delta E < 0 \\ \exp(-\frac{\Delta E}{T}), & \text{if } \Delta E \geq 0 \end{cases} \quad (5)$$

where $\Delta E = F(S_{\text{new}}) - F(S_{\text{current}})$.

2) **Key Features: Two-Level SA Architecture:** The SA implementation using a hierarchical two-level structure for neighborhood generation:

- **Inner SA (every 25 iterations):** Task inclusion layer enabled by setting `include_unassigned=True`. For *each car*, one of three actions is chosen uniformly at random:
 - +1: Assign one currently unassigned task to the car (if available), appending it to the end of its task sequence with a newly generated path.
 - 0: No change is applied to this car.
 - -1: Unassign a randomly selected task from the car, return it to the unassigned task pool, and rebuild the remaining paths for that car.

This layer allows the solution to dynamically expand or contract its task set.

- **Outer SA (all other iterations):** A single assigned task is randomly selected and moved from its current car to a different car. The task is appended to the end of the new car's task list with a regenerated path, while the paths for the original car are rebuilt to maintain feasibility. This layer strictly redistributes existing tasks without introducing or removing tasks from the solution.

Priority-Based Task Insertion: Tasks are ordered by priority, ensuring high-priority tasks are attempted first.

Path Connectivity: After modifications, paths are rebuilt to ensure continuity:

$$\text{end}(S_{i,j}) = \text{start}(S_{i,j+1}) \quad (6)$$

B. Genetic Algorithm

The GA follows standard steps: initialization of a random population, fitness evaluation, tournament selection, crossover, mutation, and iterative repetition until convergence.

1) **Nested GA Architecture:** The GA implementation uses a hierarchical two-level structure:

Outer GA (Task Assignment): Evolves a population of $N = 20$ feasible task-to-Car assignments using tournament selection (size 2), uniform crossover (40%), and car assign/unassign mutation (50%).

Inner GA (Path Optimization): For each outer solution, evolves $N = 15$ path variations to minimize total distance using elitism (10%), 2-point crossover, and three mutation operators:

- **Waypoint Perturbation (67%):** Perturbs 1-2 waypoints to adjacent cells
- **Segment Regeneration (33%):** Regenerates path segments with adjacency validation
- **Feasibility Enforcement:** Solutions violating collision or distance constraints are repaired through greedy task removal

Termination: Outer GA stops after 50 generations ($G_{\max}$) or 15 generations without improvement (G_{conv}). Inner GA runs for 10 iterations per outer solution evaluation.

This nested architecture separates task assignment optimization from path planning, allowing focused optimization at each level while maintaining feasibility through repair mechanisms.

C. Discrete Grey Wolf Optimizer

1) **Two-Level D-GWO Architecture:** Similar to the nested GA approach, D-GWO employs a hierarchical structure:

Outer D-GWO (Task Assignment): Evolves a pack of $N = 20$ wolves through leader-guided movements. The three best solutions (alpha, beta, delta) guide remaining wolves using discrete operators adapted for task reassignment.

Inner D-GWO (Path Optimization): For each outer solution, evolves path variations over 10 iterations to minimize distance using:

- **Hamming Distance Movement:** Wolves move toward leader paths using discrete distances tailored to task assignment space
- **Mixed Path Operators:** Waypoint perturbation, segment regeneration, and leader waypoint adoption
- **Feasibility Enforcement:** Solutions are checked for adjacency and distance constraints with repair mechanisms

2) **Key Innovations: Discrete Movement Operators:** Continuous GWO updates replaced with Hamming-distance-based discrete movements for task assignment sequences.

Leader-Guided Path Adjustments: Wolves adopt intermediate waypoints from leaders for smoother path convergence.

These adaptations enable D-GWO to operate in discrete, constraint-heavy domains while preserving exploration-exploitation balance.

D. Artificial Bee Colony (ABC) Optimizer

1) *Two-Level ABC Architecture*: Similar to the nested GA approach, ABC employs a hierarchical structure with two specialized components:

Outer ABC (Task Assignment): Maintains a colony of N food sources representing different task-to-car assignments. Three types of bees cooperate: *employed bees* exploit current food sources, *onlooker bees* probabilistically select promising sources, and *scout bees* explore new regions when sources are abandoned.

Inner ABC (Path Optimization): For each outer food source, maintains path variations with identical task assignments but different waypoint sequences. Optimizes routes using discrete path operators while preserving task allocation structure.

2) *Food Source Representation*: Each food source encodes:

- **Task Assignment Matrix**: Car-to-task allocation
- **Task Execution Order**: Sequence for each car
- **Path Waypoints**: Spatial routes between tasks

3) *Employed Bee Phase*: Each employed bee generates a neighbor solution by modifying its food source:

Neighbor Generation Operators:

- *Task Swap*: Exchange tasks between two different cars
- *Task Reassignment*: Move task from one car to another
- *Order Swap*: Change execution sequence within car
- *Unassigned Inclusion*: Assign previously unassigned tasks

Greedy Selection: If neighbor fitness $F(S_{new}) < F(S_{current})$, replace food source and reset trial counter; otherwise increment trial counter.

4) *Onlooker Bee Phase*: Onlooker bees select food sources using fitness-proportionate selection:

$$P_i = \frac{\max(F) - F_i + \epsilon}{\sum_{j=1}^N (\max(F) - F_j + \epsilon)} \quad (7)$$

where P_i is the selection probability for food source i , and $\epsilon = 10^{-10}$ prevents division by zero. Selected sources undergo neighbor generation similar to employed bees.

5) *Scout Bee Phase*: Food sources are abandoned when trial counter exceeds limit L :

Abandonment Criterion: If $\text{trials}_i \geq L$, replace with new random feasible solution.

Scout Generation: Creates new food source through random task assignment with feasibility validation (max 1000 attempts), applying solution repair if necessary.

6) *Inner ABC Path Operators*: For fixed task assignments, path optimization employs:

Waypoint Perturbation: Randomly modify intermediate waypoints within adjacent cell constraints (8-connectivity).

Segment Regeneration: Replace path segments between consecutive tasks with newly generated random paths.

Path Crossover: Exchange path segments between two food sources while maintaining endpoint connectivity.

7) *Feasibility Enforcement*: Solutions undergo multi-stage validation:

- 1) **Distance Constraints**: Remove tasks exceeding max_distance
- 2) **Path Connectivity**: Ensure all waypoints are adjacent (8-connected)
- 3) **Map Boundaries**: Validate all positions within grid limits
- 4) **Sequential Paths**: Regenerate paths to ensure proper task-to-task transitions

Infeasible solutions receive penalty: $F_{\text{penalty}} = F + 10 \cdot \max(F_{\text{feasible}})$.

8) *Key ABC Optimizations: Nested Architecture Integration*: Outer ABC invokes inner ABC after each neighbor generation (employed/onlooker phases) to optimize paths for modified task assignments.

Diversity Preservation: Colony diversity measured using normalized Hamming distance between task assignments:

$$D = \frac{1}{|T|} \cdot \frac{2}{N(N-1)} \sum_{i=1}^{N-1} \sum_{j=i+1}^N H(S_i, S_j) \quad (8)$$

where $H(S_i, S_j)$ is Hamming distance, $|T|$ is number of tasks, N is colony size.

Adaptive Exploration: Scout bee mechanism automatically balances exploitation (employed/onlooker) and exploration (scout) by monitoring solution improvement stagnation.

Solution Repair: Greedy distance-based repair removes tasks sequentially until car distance constraints are satisfied, preserving maximum task completion.

9) *Convergence Criteria*: Optimization terminates when:

- Maximum cycles reached, OR
- No fitness improvement for K_{conv} consecutive cycles

The algorithm returns the best food source found, representing optimal task-to-car assignment with minimized path distances.

V. RESULTS

A. Test Problem Instances

Three test cases with varying complexity levels were designed to evaluate algorithm performance comprehensively. Weather zones and task placements are assigned randomly across the grid; vehicle starting positions are fixed (static).

1) *Test Case Specifications*: **Case 1 (Medium)** represents the baseline scenario with moderate constraints, **Case 2 (Low)** corresponds to a small-scale problem under favorable conditions, and **Case 3 (High)** describes a large-scale scenario characterized by challenging conditions.

B. Solution Visualization

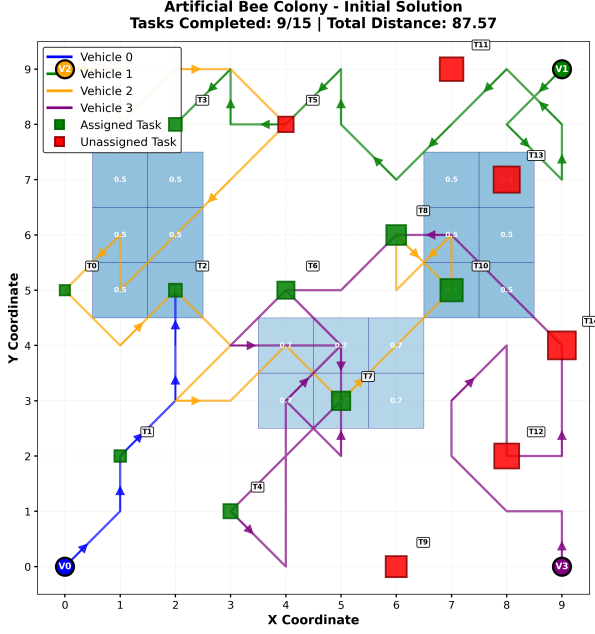


Fig. 1: Initial Solution (Case 1): 9/15 tasks completed, total distance 87.57

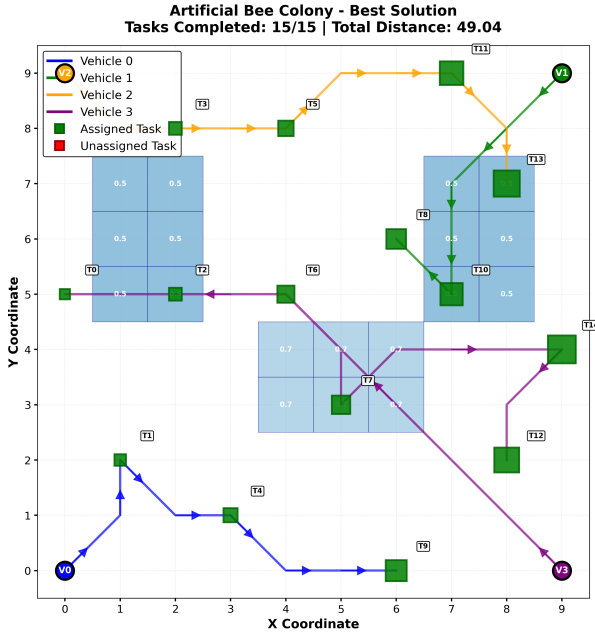


Fig. 2: Optimized Solution by ABC (Case 1): 15/15 tasks completed, total distance 49.04

A comparison between the initial solution (Figure 1) and the optimized solution obtained by the ABC algorithm (Figure 2) highlights significant performance improvements in Case 1. Task completion increased from 9 to 15 tasks (67%), achieving full coverage, while the total traveled distance was reduced from 87.57 to 49.04 units (44% reduction). Overall, this resulted in a 92.87% improvement in fitness value, confirming

the effectiveness of the algorithm in multi-objective optimization.

C. Test Case Characteristics

Three test scenarios were constructed with distinct problem parameters to assess algorithm scalability and robustness. Each case defines a search-and-rescue environment with a specific map size, number of vehicles, number of tasks, and weather conditions, ranging from low to high complexity.

TABLE I: Comparison of Test Case Characteristics

Characteristic	Case 1	Case 2	Case 3
Map Size	10×10	8×8	15×15
Vehicles	4	3	6
Tasks	15	8	25
Tasks per Vehicle	3.75	2.67	4.17
Weather Zones	18	0	135
Complexity	Medium	Low	High

D. Performance Results

Each algorithm was applied to all three test cases starting from the same initial solution. The table reports the number of completed tasks and the total travel distance before and after optimization, showing the improvement achieved by each algorithm per test case.

TABLE II: Performance Results Across All Test Cases (Initial→Best)

Case	Metric	GA	SA	GWO	ABC
1	Completed Tasks	9→13	9→9	9→14	9→15
	Total Distance	87.57→47.04	87.57→87.57	87.57→48.63	87.57→49.04
2	Completed Tasks	7→8	7→7	7→8	7→8
	Total Distance	69.18→21.49	69.18→54.28	69.18→19.49	69.18→17.90
3	Completed Tasks	13→16	13→13	13→21	13→24
	Total Distance	142.02→73.36	142.02→142.02	142.02→83.18	142.02→102.33

E. Comparative Analysis

The following table (Table III) summarizes the aggregate performance of each algorithm across multiple independent runs on the benchmark test cases. The best fitness, mean fitness, standard deviation, and average execution time were recorded to enable a direct comparison of solution quality, consistency, and computational cost.

TABLE III: Performance Comparison of Optimization Algorithms (Case 1) – Ranked Best to Worst

Rank	Algorithm	Best Fitness	Mean Fitness	Std Dev	Mean Time (s)
1	ABC	49.04	52.49	2.69	738.70
2	GWO	42.80	146.46	64.75	3674.69
3	GA	57.28	233.22	128.92	66.99
4	SA	687.57	835.22	89.92	0.17

GWO requires coordinated position updates across all agents relative to three leaders (α , β , δ) each iteration, incurring more overhead than SA, ABC, and GA.

All algorithms were evaluated under identical hardware conditions to ensure a fair and unbiased comparison. The experiments were conducted on a laptop equipped with an AMD Ryzen 7 5800H processor, an NVIDIA GeForce RTX 3050 Ti GPU, 24 GB of RAM, and SSD storage. Each algorithm was executed over 10 independent runs per algorithm, and the reported results were obtained from these

runs for each algorithm individually. The evaluation metrics used are as follows: **Best Fitness** measures the optimality of the best solution found, with lower values indicating better performance; **Standard Deviation (Std Dev)** measures the repeatability and stability of the algorithm across multiple runs, where lower values indicate more consistent behavior; and **Mean Time** measures the computational complexity and efficiency of each algorithm, reflecting the average wall-clock time required per run.

F. Algorithm Convergence Analysis

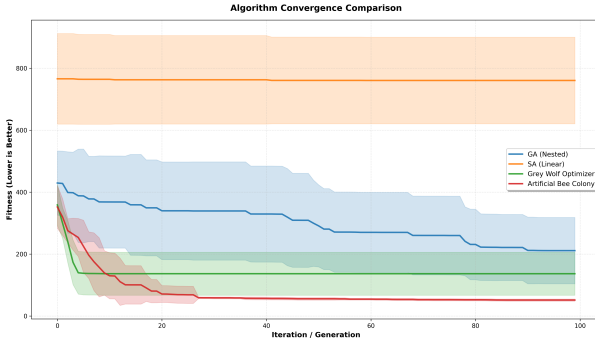


Fig. 3: Convergence behavior over 100 iterations (Case 1)

The convergence curves (Figure 3) reveal distinct algorithmic behaviors on Case 1. ABC demonstrates rapid convergence within 15 iterations to fitness values around 52 with minimal variation ($\text{std}=2.69$), as evidenced by its narrow confidence band. GWO shows quick initial improvement to approximately 146 but plateaus early with limited further optimization. GA exhibits gradual, continuous improvement from high initial values down to approximately 233 over all generations with increasing variability ($\text{std}=128.92$). SA displays poor performance with near-zero improvement, maintaining fitness values around 835 throughout. These patterns directly corroborate the statistical results in Table III, confirming ABC’s superior convergence speed, solution quality, and consistency.

G. Summary of Algorithm Performance

Based on the quantitative results from the preceding experiments, the following table (Table IV)provides a qualitative summary of the key strengths and weaknesses observed for each optimization algorithm across all test cases.

TABLE IV: Algorithm Strengths and Weaknesses

Algorithm	Strengths	Weaknesses
GA	100% feasibility; achieves full task completion in low-complexity cases; moderate computational time (66.99s)	High variability ($\text{std}=128.92$); inconsistent performance across runs
SA	Fastest execution time (0.17s)	Poor feasibility (60% Case 1, 20% Case 2, 0% Case 3); no optimization improvement; worst solution quality. This is due to SA’s inability to improve/recover from a bad solution.
GWO	100% feasibility; second-best mean fitness (146.46); good performance in high-complexity cases	Significantly higher computational time (3674.69s)
ABC	Best mean fitness (52.49); lowest variance ($\text{std}=2.69$); 100% feasibility; full task completion; most consistent performance	Higher computational cost than GA (738.70s)

VI. CONCLUSION AND FUTURE WORK

This study evaluated four algorithms (GA, SA, GWO, ABC) for multi-vehicle search-and-rescue routing under weather, collision, and task constraints.

Results prove the Artificial Bee Colony (ABC) algorithm is the most robust solution. ABC achieved a 100% feasibility rate, optimal task completion, and rapid convergence within 15 iterations. In contrast, Simulated Annealing (SA) failed entirely in complex environments, Grey Wolf Optimizer (GWO) was computationally impractical (3674s/run), and the Genetic Algorithm (GA) proved too inconsistent.

Future development will discard unscalable models to build an ABC-driven hybrid engine. The focus will shift to real-time weather integration, dynamic task insertion, and slashing the current 738s computational cost to enable true real-time deployment for ultra-large disaster environments.

REFERENCES

- [1] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by simulated annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [2] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: University of Michigan Press, 1975.
- [3] K. Panwar and K. Deep, “Discrete grey wolf optimizer for symmetric travelling salesman problem,” *Applied Soft Computing*, vol. 105, p. 107298, 2021.
- [4] D. Karaboga and B. Basturk, “A powerful and efficient algorithm for numerical function optimization: Artificial bee colony (abc) algorithm,” *Journal of Global Optimization*, vol. 39, no. 3, pp. 459–471, 2007.
- [5] X. Zhao, R. Li, and T. Wang, “A two-stage optimization framework for emergency logistics using gwo and cbba,” *Drones*, vol. 9, no. 7, 2025.
- [6] “Distributionally robust and learning-based vehicle routing for disaster blood logistics,” *IEEE Access*, 2023.
- [7] C. Liu *et al.*, “Dynamic real-time multi-uav cooperative mission planning method under multiple constraints,” *arXiv preprint arXiv:2506.02365*, 2025.
- [8] A. Zhang, Y. Zhang, and W. Gong, “Robust optimization of electric vehicle paths in uncertain environments,” *IEEE Access*, vol. 11, 2023.
- [9] L. Zhao, Q. Feng, and S. Yuan, “A high-efficiency task allocation algorithm for multiple uavs in offshore wind power under energy constraints,” *Journal of Marine Science and Engineering*, vol. 13, no. 9, p. 1711, 2025.
- [10] J. Zhang, Y. Liu, and M. Chen, “Uav-ugv cooperative trajectory optimization for post-disaster medical rescue,” *World Scientific*, 2025.
- [11] S. Banerjee, A. Gupta, and P. Das, “Rich vehicle routing problem for disaster management with multi-modal transshipment,” *arXiv preprint arXiv:2509.13227*, 2025.
- [12] F. Ahmed, H. Amr, M. Khalil, M. Mohamed, Y. Amir, Z. Yasser, J. Magdy, and O. M. Shehata, “Route to clean cities: Waste collection optimization with uavs,” in *2025 15th International Conference on Electrical Engineering (ICEENG)*. Cairo, Egypt: IEEE, May 2025, pp. 1–6.
- [13] A. H. Ghoneim, A. M. ElSoda, H. A. Abdelmawla, K. Elbadawy, L. Tamer, L. Tarek, J. Magdy, and O. M. Shehata, “Metaheuristic optimization for multi-agv path planning in search and rescue missions,” in *2025 15th International Conference on Electrical Engineering (ICEENG)*. IEEE, May 2025, pp. 1–6.
- [14] H. Tang, X. Liu, J. Wang, and Y. Zhao, “Improved pso-based two-phase logistics uav path planning under dynamic demand and wind conditions,” *Drones*, vol. 8, no. 8, p. 356, 2024.
- [15] Y. Wang, L. Chen, and K. Zhang, “Wind-aware uav path planning using simulated annealing,” *arXiv preprint arXiv:2403.12876*, 2024.
- [16] K. Chan, D. Wong, and F. Li, “Wind dynamic and energy-efficiency path planning for unmanned aerial vehicles in the lower-level airspace and urban air mobility context,” *Applied Energy*, vol. 345, p. 120011, 2023.
- [17] P. Kumar and R. Roy, “Wind-resilient path planning for uavs using potential fields and meta-heuristic optimization,” in *Proc. IEEE Int. Symp. Safety, Security, Rescue Robotics (SSRR)*. IEEE, 2018, pp. 1–7.
- [18] W. Li, H. Zhang, and Z. Wu, “Wind-adaptive lifelong planning a* for uavs in dynamic environments,” *Aerospace*, vol. 9, no. 12, p. 782, 2024.